LINFO1361: Artificial Intelligence
Assignment 1: Local Search and CSP.

Alice Burlats, Achille Morenville, Thomas Bouvencourt, Amaury Fierens, Eric Piette
March 2026

Guidelines

- This assignment is due on **Wednesday, 18th March 2026, 18h00**.
- *No delay* will be tolerated.
- *Document* your source code (at least the difficult or more technical parts of your programs). Python docstrings for important classes, methods and functions are also welcome.
- Copying code or answers from other groups (or from the internet) is strictly forbidden. Each source of inspiration must be clearly indicated.
- Source code shall be submitted on the online *INGInious* system. Only programs submitted via this procedure will be graded. No program sent by email will be accepted.
- Respect carefully the *specifications* given for your program (arguments, input/output format, etc.) as the program testing system is *fully automated*.
- The answer to questions must be given by filling in the latex template provided. The final file must be submitted on *gradescope*. No report sent by email will be accepted.
- Nothing must be modified in the template except your answer that you insert in the *answer* environments as well as your names and your group number. The names are provided through the command *students* while the command *group* is used for the group number. The dimensions of *answer* fields *must not* be modified either. For the tables, put your answer between the "&" symbols.
- To submit on gradescope, go to `https://gradescope.com` and click on the "log in" button. Then choose the "school credentials" option and search for *UCLouvain Username*. Log in with your global username and password. Find the course LINFO1361 and the Assignment 1, then submit your report. Only one member should submit the report and add the second as group member.
- Check this link if you have any trouble with group submission `https://help.gradescope.com/article/m5qz2xsnjy-student-add-group-members`

Requirements

For this assignment, you will use the python library **PyCSP3**. Install it from pip:

```
$ pip install pycsp3==2.4
```

You can find more information here. **Warning: You need to install the version 2.4 of PyCSP3. The version 2.5 change the output format, and you may encounter issues when submitting**

your code on INGInious.

1 Understanding a CP model

The sudoku is a well-known puzzle game. The goal is to complete a partially filled 9×9 grid with digits between 1 and 9 such as:

- Each row contains all digits from 1 to 9.
- Each column contains all digits from 1 to 9.
- Each of the nine 3×3 subgrids contains all digits from 1 to 9.
- The values given in the initial grid need to be respected.

	6	2	5				7	
	8	5		6	7			9
					9			
	3					9	8	4
			1	3				
	2				5	6	1	
4	9		7					
		8	6	9			4	
7						1	9	

Figure 1: Sudoku instance

An example of a sudoku instance is shown in figure 1.

The file `sudoku.py` contains a model to solve the sudoku instance shown in figure 1. You can run it by using the following command in a terminal:

```
python3 sudoky.py
```

Task

The model contains several set of constraints. Your task for this exercise is to read and understand this model. You can then answer the MCQ available on INGInious.

Next, we want this model to be able to solve a variant of the sudoku problem: all the rule of sudoku are kept but a new constraint is added: the center of each 3×3 subgrid (i.e., the cells $(1,1)$, $(1,4)$, $(1,7)$, $(4,1)$, $(4,4)$, $(4,7)$, $(7,1)$, $(7,4)$ and $(7,7)$) needs to be different. You thus need to add a supplementary constraint to the given model and submit it on INGInious.

Questions

1. **0.5pt** Answer correctly to the questions 1-4. You can help yourself by looking at `https://www.pycsp.org/documentation/constraints/` to find the definition of the different constraints.
2. **0.5pt** Answer correctly to the question 5.
3. **1pt** Add the needed constraint to the model and submit it on INGInious.

   Warning ! The MCQ allows only one submission. So be careful when submitting your answers.

2 Minesweeper

In the famous game of minesweeper, the goal is to find the position of hidden mines in a grid. To help the player, cells adjacent to mines contain a number corresponding to the number of mines in their neighborhood. In the original game, these clues are hidden at the beginning of the game, but here we consider that they are all given as input. The goal is to find the position of the mines among the empty cells of the grid.

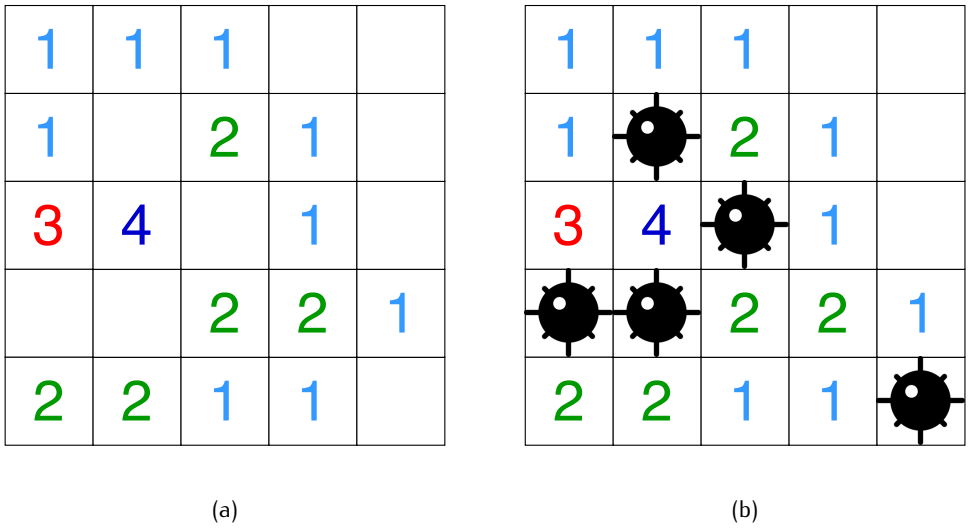


Figure 2: Example of a Minesweeper problem instance. On left, the instance. On right its solution.

2.1 Input

An instance is defined by an $n \times m$ grid of integer. Empty cells are represented by -1 while clues are represented by a number between 0 and 8 corresponding to the number of mines in their neighborhood. **Mines can only be placed on empty cells.**

For example, the instance shown in figure 2a is represented by the following matrix:

$$\begin{bmatrix} 1 & 1 & 1 & -1 & -1 \\ 1 & -1 & 2 & 1 & -1 \\ 3 & 4 & -1 & 1 & -1 \\ -1 & -1 & 2 & 2 & 1 \\ 2 & 2 & 1 & 1 & -1 \end{bmatrix}$$

2.2 Output

You are supposed to return the position of the mines as a list of coordinates of the form (i, j) where i is the row number and j is the column number of a cell containing a mine. For example, the solution of the instance shown in figure 2b is given by the following list of coordinates: $[(1, 1), (2, 2), (3, 0), (3, 1), (4, 4)]$.

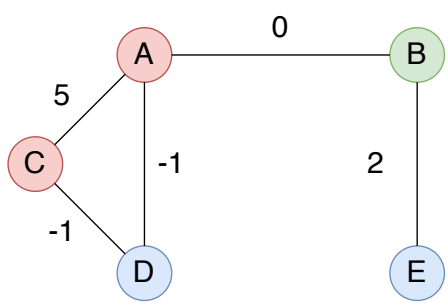
Warning : Not all given input can lead to a valid solution. In the case of an unsatisfiable input, you should return None.

Questions

1. **4pt** Implement your CP model in the file `minesweeper.py` and submit it on INGInious.
2. **4pt** Describe your CP model: First describe the variables of your CP model. For each set of variables, explain what they represent, give their domains and how many variables are in this set. Then, list the constraints of your CP model. For each constraint explain what it enforces, and on which variables it is applied.

3 Atom Placement Problem

This exercise deals with a simplification of a physics problem aiming at finding the ground state of a crystal. Your goal is to place a given set of atoms among provided sites. Atoms needs to be placed in such a way to minimize the energy associated to the interaction between them.



H

	Type 0	Type 1	Type 2
Type 0	5	0	-1
Type 1	0	7	2
Type 2	-1	2	4

$s_A = 0; s_B = 1; s_C = 0; s_D = 2; s_E = 2$

Energy = 5 - 1 - 1 + 0 + 2 = 5

Figure 3: Example of an atom placement problem

You are provided with graph $G(V, F)$ where V corresponds to a set of sites where atoms will be placed and F is a set of edges. The graph is simple, unweighted and non-directed. If

there is an edge $(i, j) \in F$, then the sites i and j are said to be neighbors. You are required to place a set of atoms N on the sites (we assume that $|N| = |V|$). There are k types of atoms. Let s_v be the type of the atom assigned on the site v. Two atoms placed in neighbor sites will interact. The energy associated to this interaction depends on the type of each node. The possible energies are given in an $k \times k$ matrix H. If two neighbors contain atoms of types i and j, the energy associated to their interaction is $H_{i,j} = H_{j,i}$ (H is symmetric). The elements of H can be positive, negative, or zero. The problem is to assign the atoms of N to the sites V in such a way as to minimize the total energy. The total energy is given by: $E = \sum_{(i,j)\in F} H_{s_i,s_j}$. E is therefore the sum of the interaction energies. Figure 3 shows an example with $k = 3$ and $N = \{0, 0, 1, 2, 2\}$.

All the atoms in N must be placed. A site can only contain one atom. A number of atoms of each type is given. Thus, if the instance says that there are 5 atoms of type 1, then the solution must contain 5 atoms of type 1, not 4 or 6.

3.1 Objective function

As explained before, the objective function is the sum of the energies of each interaction :

$$E = \sum_{(i,j)\in F} H_{s_i,s_j} \tag{1}$$

where the set of variables to assign is $s = \{s_0, s_1, s_2, .., s_{|V|-1}\}$.

The solution must follow the constraints :

$$\forall i \in V : s_i \in \{0, \cdots, k-1\} \tag{2}$$

and

$$\forall a \in \{0, \cdots, k-1\} : n_a(s) = n_a(N) \tag{3}$$

where $n_a(A)$ is the number of occurrence of value a in set A.

3.2 Neighborhood

In local search, we have to build at each decision time a neighborhood in which a solution is picked depending on a specific criterion. We suggest to only consider local solutions with exactly $|V|$ vertices, and to use a simple swap action to build the neighborhood. This action consists of the swap of two vertices u and v where $s_u \neq s_v$.

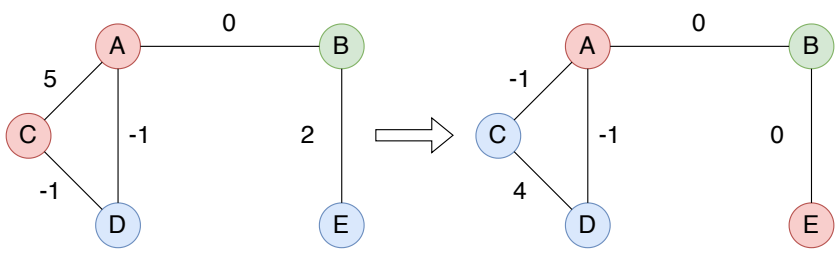


Figure 4: Swap of two vertices

You are not obliged to use the proposed objective function as well as techniques for building the neighborhood. You can use your owns but you have to specify in your report

the ones you used. However, your solution has to contains the required number of atoms of each type. Moreover, note that your result will be evaluated with the function described in 1.

The format for describing the different instances on which you will have to test your program is the following:

Input format

$$
\begin{array}{llll}
n & k & f & \\
 & & & \\
n_0 & n_1 & \dots & n_{k-1} \\
 & & & \\
H_{0,0} & H_{0,1} & \dots & H_{0,k-1} \\
H_{1,0} & H_{1,1} & \dots & H_{1,k-1} \\
\dots & & & \\
H_{k-1,0} & H_{k-1,1} & \dots & H_{k-1,k-1} \\
 & & & \\
u_0 & v_0 & & \\
u_1 & v_1 & & \\
\dots & & & \\
u_{f-1} & v_{f-1} & &
\end{array}
$$

An atom placement instance contains $2 + k + |F| + 3$ lines. The instance is divided in four sections, separated by a blank line The first line contains 3 integers :the number of atoms/sites $n = |N|$, the number of types k and the number of edges $f = |F|$. The second section contains the number of atoms of each types (n_l is the number of atoms of type l). The following section, containing k lines, gives the H matrix. Each line contains k integers. The ith value of the jth row is the energie between type i and j. And the last section, which contains f lines, present each edge in F : each line contains two integers that are the indexes of the nodes that are linked by this edge.

3.3 Diversification versus Intensification

The two key principles of Local Search are intensification and diversification. Intensification is targeted at enhancing the current solution and diversification tries to avoid the search from falling into local optima. Good local search algorithms have a tradeoff between these two principles. For this part of the assignment, you will have to experiment this tradeoff.

3.4 Implementation

A partial implementation is given in the `atom_placement` directory:

- *atom_placement_state.py* contains the definition of the state of the problem.
- *atom_placement.py* contains the definition of the problem and the methods to generate the states and to evaluate them.
- *lsnode.py* contains the definition of the node used in the local search.
- *search.py* contains the implementation of the different local search algorithms.
- *main.py* loads the instances and runs the local search algorithms.

3.5 Tasks

For this assignment, you will use *Local Search* to find good solutions to the atom placement problem. The test instances can be found on Moodle.

Your first task is to implement the methods `AtomPlacement.neighbors(state)` and `AtomPlacement.value(state)` in the *atom_placement.py* file, which respectively return the neighbors of a given state and the value of a given state. You can implement these methods following the suggestions given in the previous sections, but you are free to implement them as you want. You can also implement other methods if you think they are useful for your implementation.

Your second task is to implement two local search strategies:

1. `maxvalue` chooses the best node (i.e., the node with maximum value) in the neighborhood, even if it degrades the quality of the current solution. The `maxvalue` strategy should be defined in a function called *maxvalue* with the following signature: `maxvalue(problem,limit=100)`.
2. `randomized maxvalue` chooses the next node randomly among the 5 best neighbors (again, even if it degrades the quality of the current solution). The `randomized maxvalue` strategy should be defined in a function called *randomized_maxvalue* with the following signature: `randomized_maxvalue(problem,limit=100)`.

You can help yourself with the *randomwalk* function defined in *search.py*.

Questions

1. **1 pt** Formulate the atom placement problem as a Local Search problem (problem, cost function, feasible solutions, optimal solutions).
2. **5 pts** Submit your code on INGInious inside the *Assignment 4 : Atom Placement* task. Your program will be evaluated on 14 instances (during 1 minute) of which 4 are hidden. We expect you to find a good enough solution on at least 12 out the 14.
3. **3 pts** Compare the 2 strategies implemented in the previous question and `randomwalk` defined in *search.py* on the given atom placement instances. Run the strategies during 100 steps, and report, in a table, the computation time, the value of the best solution and the number of steps needed to reach the best result. For the randomized max value and the random walk, each instance should be tested 10 times to reduce the effects of the randomness on the result. When multiple runs of the same instance are executed, report the mean of the quantities.
4. **4 pts** Answer the following questions:
   (a) What is the best strategy?
   (b) Why do you think the best strategy beats the other ones? What conclusions can be drawn on the atom placement problem ?
   (c) What are the limitations of each strategy in terms of diversification and intensification?

(d) What is the behavior of the different techniques when they fall in a local optimum?